

0.91. Le titre, puis le jeu

On réunit l'écran titre (le 0.90) et le jeu du 0.81 : **START** fait disparaître le titre, pose le P, et la partie commence.

1. Ce qu'il faut comprendre

Deux programmes qu'on connaît, réunis en un : l'écran titre (le 0.90.1) et le **jeu de la pièce** (le 0.81 : le A suit la croix et ramasse le P).

Ce qui est nouveau ici : le jeu n'existe qu'après START. Tout ce que faisait la boucle du 0.81 est rangé dans `if (ecran == 1) { ... }`. Sur le titre (`ecran == 0`), le A n'est pas là, la croix ne fait rien.

Ce que le 0.81 faisait avant la boucle (poser le P, écrire SCORE) se fait maintenant **au moment de START** : c'est là que l'écran du jeu commence.

Deux fois `if`, pas `if ... else`, pour rester comme au 0.90.1 : le premier regarde START sur le titre, le second fait tourner le jeu.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <couleurTexte>` → [0.87.7. La fonction couleurTexte\(\) — la couleur des lettres](#)
- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1809 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	93 o	← la console, à l'allumage — la mise en route de la console
texte_grand_0()	195 o	← main() — ta fonction
deplace_croix()	217 o	← main() — une fonction fournie par le langage
viderEcran()	33 o	← main() — ta fonction
main()	156 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases

```

EcrireTexte      10 o ← texte() – écrit un texte, lettre par lettre
EcrireNombre     33 o ← nombre() – écrit un nombre en chiffres
CopierTuiles     18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte     14 o ← démarrage – vide les deux cartes de l'écran
LireManette      37 o ← bouton() – lit les huit boutons
Tuiles           848 o ← CopierTuiles – la police entière, et 9 tuiles dessinées
"APPUIE SUR START" 16 o ← texte() – un texte gravé
"SCORE"          5 o ← texte() – un texte gravé
GRAND_TABLE_0_0_0 12 o ← poser() – une table gravée
GRAND_TABLE_0_1_1 12 o ← poser() – une table gravée
GRAND_TABLE_0_2_2 12 o ← poser() – une table gravée
GRAND_TABLE_0_3_3 12 o ← poser() – une table gravée
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
    poser()      × 6    148 o sur place, dans texte_grand_0(), deplace_croix(), main() + AdresseCase (10 o),
GRAND_TABLE_0_0_0 (12 o), GRAND_TABLE_0_1_1 (12 o), GRAND_TABLE_0_2_2 (12 o), GRAND_TABLE_0_3_3 (12 o)
    effacer()    × 2    29 o sur place, dans deplace_croix(), viderEcran() + AdresseCase (10 o), EffacerCases (12
o)
    bouton()     × 5    25 o sur place, dans deplace_croix(), main() + LireManette (37 o)
    texte()      × 2    22 o sur place, dans main() + EcrireTexte (10 o)
    couleurFond() × 1    12 o sur place, dans main()
    nombre()     × 1    10 o sur place, dans main() + EcrireNombre (33 o)
    image()      × 1    3 o sur place, dans main() + AttendreImage (28 o)
    images()     × 1    2 o sur place, dans deplace_croix()
Tes fonctions appelées :
    deplace_croix() × 1    19 o sur place, dans main()
    texte_grand_0() × 1    3 o sur place, dans main()
    viderEcran()   × 1    3 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <effacer>           // efface des cases, ou tout le fond
#include <couleurTexte>      // choisit la couleur des lettres
#include <textGrand>         // écrit un texte agrandi, de 1 à 20 fois
#include <text>              // écrit un texte à l'écran
#include <bouton>            // lit un bouton de la manette
#include <poser>             // pose une tuile sur une case du fond
#include <ALPHABET>          // les lettres de la police : ALPHABET[0] est le A
#include <deplace_croix>     // une tuile qui suit la croix, case par case
#include <nombre>            // écrit un nombre en chiffres

```

2. UNE VARIABLE — ECRAN

```

uint8_t ecran = 0;          // 0 = le TITRE, 1 = le JEU

// Les variables du jeu du 0.81 :

```

3. UNE VARIABLE — X

```

uint8_t x = 5;              // le A : sa colonne...

```

4. UNE VARIABLE — Y

```

uint8_t y = 8;              // ...et sa ligne (le 0.81)

```

5. UNE VARIABLE — PX

```

uint8_t px = 15;            // la pièce, le P : sa colonne...

```

6. UNE VARIABLE — PY

```

uint8_t py = 8;             // ...et sa ligne

```

7. UNE VARIABLE — SCORE

```
uint8_t score = 0;          // les pièces ramassées
```

8. UNE FONCTION — VIDERECRAN()

```
void viderEcran() {  
    for (uint8_t l = 0; l < 18; l++) {  
        effacer(0, l, 20);  
    }  
}
```

9. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    couleurTexte(31, 16, 0);          // le titre du 0.90  
    texteGrand(4, 5, "JEU", 4);  
    texte(2, 12, "APPUIE SUR START");  
  
    while (true) {  
        image();  
  
        // ---- ÉCRAN 0 : LE TITRE. START : on prépare le jeu, une seule fois.  
        if (ecran == 0 && bouton(START)) {  
            ecran = 1;  
            viderEcran();  
            poser(px, py, ALPHABET[15]);    // le P (ce que le 0.81 faisait AVANT la boucle)  
            texte(0, 17, "SCORE");  
        }  
  
        // ---- ÉCRAN 1 : LE JEU. La boucle du 0.81, seulement sur cet écran.  
        if (ecran == 1) {  
            deplace_croix(x, y, ALPHABET[0], 250);    // le A suit la croix  
            if (x == px && y == py) {                  // le A sur le P : ramassée (le 0.81)  
                score = score + 1;  
                px = 20;                                // la pièce part hors de l'écran  
            }  
            nombre(6, 17, score);  
        }  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



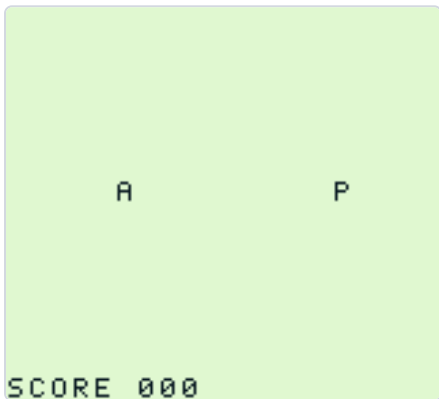
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



START — l'écran a changé

5. Ce qu'on doit voir

Le titre ; START : le titre disparaît, le A et le P apparaissent, et on ramasse le P avec la croix.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ sur le titre, pas de pièce
- ✓ après START, le P est posé
- ✓ on le ramasse : le score vaut 1

Ensuite : **0.91.1. Le titre, puis le jeu — trente secondes** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++